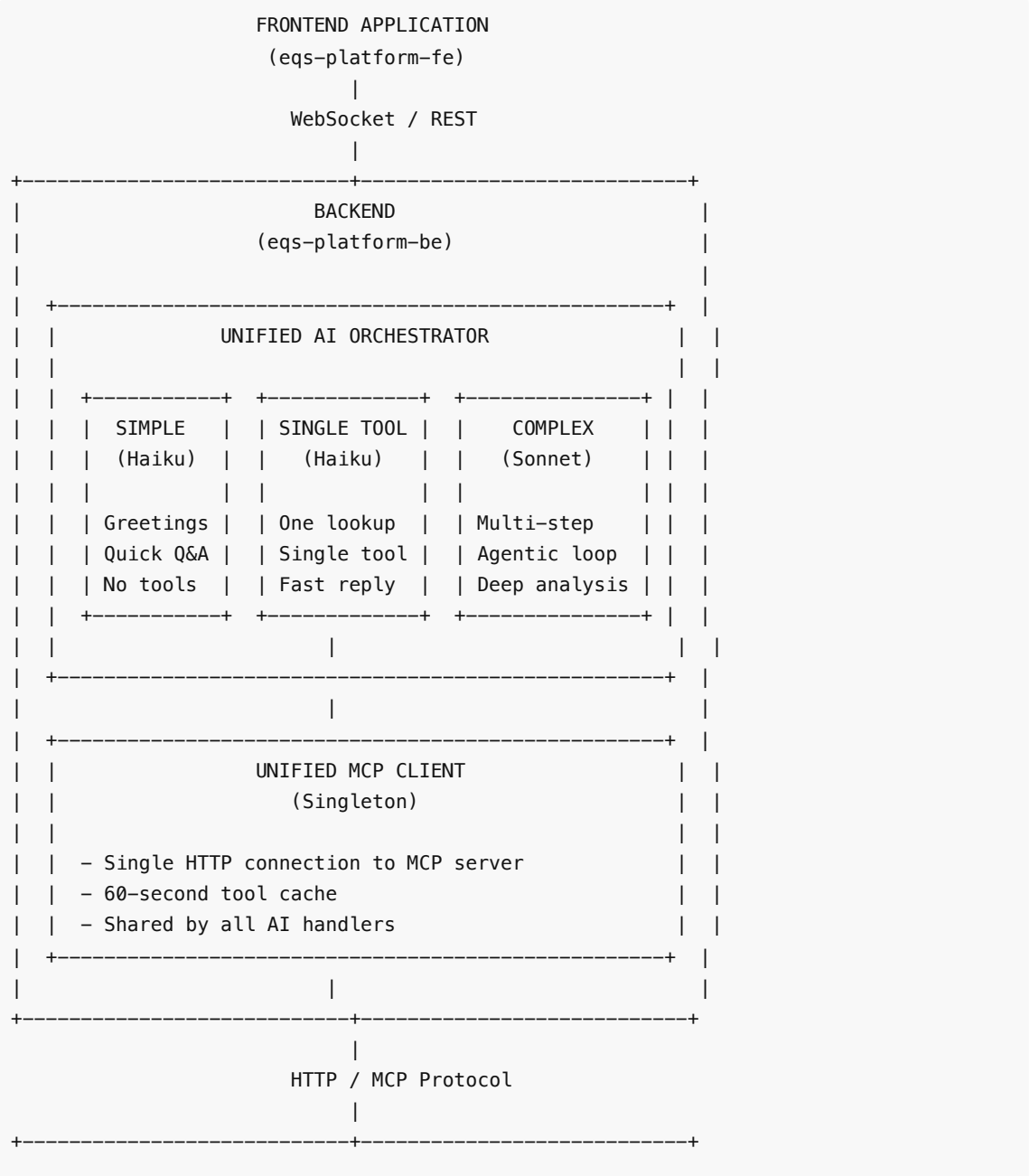
Unified AI Orchestrator Architecture

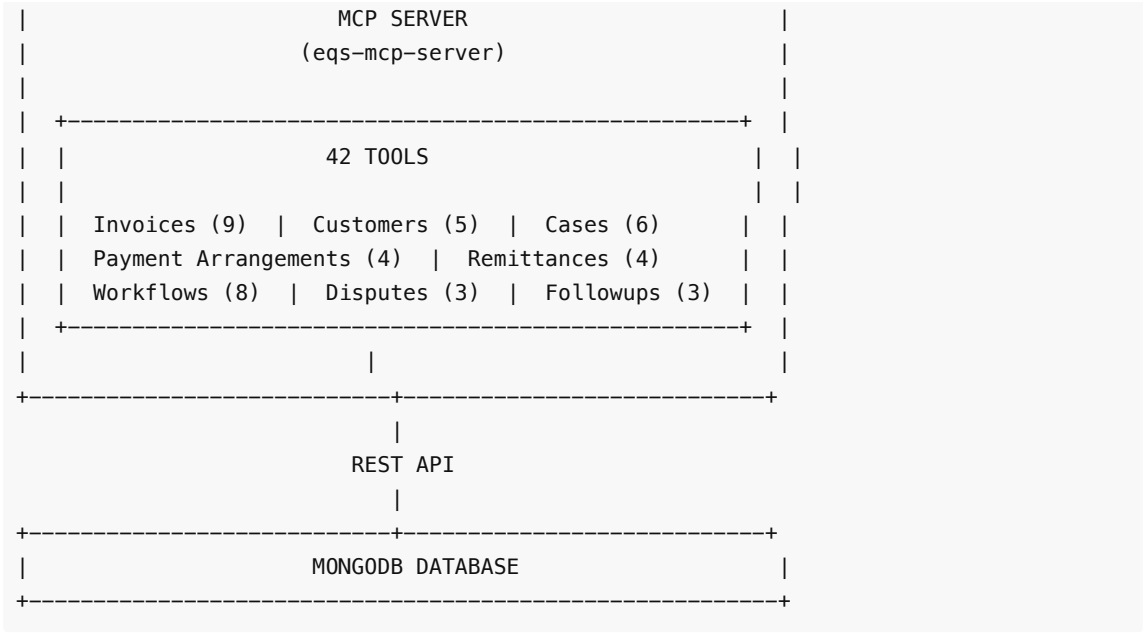
Version: 2.0 **Date:** February 2026 **Branch:** eqs-114

Executive Summary

The EQS Platform implements a **Unified AI Orchestrator** pattern based on Anthropic's recommended architecture for agentic systems. This document describes the architecture following the recent unification of the chatbot and agentic engine into a single orchestration layer.

Architecture Overview





Complexity Classification

The orchestrator classifies every incoming request into one of three complexity levels:

Complexity	Model	Use Case	Max Iterations
Simple	Claude 3.5 Haiku	Greetings, general questions	1
Single Tool	Claude 3.5 Haiku	One specific lookup	1-2
Complex	Claude Sonnet 4	Multi-step analysis	Up to 10

Classification Logic

```
// 1. Check for simple greetings
"hi", "hello", "thanks" → SIMPLE

// 2. Check for complex indicators
"analyze", "compare", "recommend", "across all" → COMPLEX

// 3. Default or LLM classification
Single entity lookup → SINGLE_TOOL
```

Request Flow

Simple Request Flow

```
User: "Hi there!"
    |
    v
```

```
+---------------------+
| Orchestrator        |
| classify() → SIMPLE |
+---------+-----------+
          |
          v
+---------------------+
| _handleSimple()     |
| - No tools          |
| - Haiku model       |
| - Quick response    |
+---------+-----------+
          |
          v
Response: "Hello! I'm EQUI AI..."
```

Single Tool Request Flow

```
User: "Show me invoice #12345"
    |
    v
+-------------------------+
| Orchestrator            |
| classify() → SINGLE_TOOL|
+---------+---------------+
          |
          v
+-------------------------+
| _handleSingleTool()     |
| - Load tools from MCP   |
| - Call Haiku with tools |
+---------+---------------+
          |
          v
+-------------------------+
| Tool: search_invoices   |
| -> MCP -> Database      |
+---------+---------------+
          |
          v
+-------------------------+
| Format response         |
| - Table with invoice    |
+-------------------------+
```

Complex Request Flow

```
User: "Analyze all overdue invoices and recommend payment plans"
    |
    v
+-------------------------+
```

```
| Orchestrator            |
| classify() → COMPLEX    |
+---------+---------------+
          |
          v
+-----------------------------------------+
| _handleComplex() - Agentic Loop         |
|                                         |
|  Iteration 1:                           |
|  -> Tool: get_overdue_trends            |
|                                         |
|  Iteration 2:                           |
|  -> Tool: search_customers              |
|                                         |
|  Iteration 3:                           |
|  -> Tool: get_customer_risk_analysis    |
|  -> Tool: get_customer_invoices         |
|                                         |
|  Iteration 4:                           |
|  -> Synthesize & Recommend              |
+---------+-------------------------------+
          |
          v
+-------------------------+
| Comprehensive response  |
| - Analysis summary      |
| - Risk breakdown        |
| - Payment plan recs     |
| - Suggested due dates   |
+-------------------------+
```

Key Components

1. Unified Orchestrator

Location: `src/services/ai-chat-bot/orchestrator.js`

```
class UnifiedOrchestrator {
  // Single entry point for all AI requests
  async processMessage(userMessage, context, streamCallback)

  // Complexity classification
  async classifyComplexity(message, context)

  // Handlers for each complexity level
  async _handleSimple(message, context, streamCallback)
  async _handleSingleTool(message, context, streamCallback)
  async _handleComplex(message, context, streamCallback)
}
```

2. Unified MCP Client

Location: `src/services/mcp/unified-mcp-client.js`

```
class UnifiedMCPClient {
  // Singleton pattern - shared by all handlers
  async connect()
  async listTools()           // Anthropic format (chatbot)
  async listToolsForAgentic() // OpenAI format (agentic)
  async executeTool(name, args, context)

  // Caching
  toolsCache          // 60-second cache
  clearCache()
}
```

3. WebSocket Handler

Location: `src/services/ai-chat-bot/websocket/chat-handler.js`

- Authenticates users via JWT
- Checks subscription access
- Enforces daily rate limits (30/day)
- Routes to orchestrator

Model Selection Strategy

Task Type	Model	Reasoning
Greetings	Haiku	Fast, cheap, no tools needed
Single lookups	Haiku	Fast response, simple tool use
Multi-step analysis	Sonnet	Better reasoning, complex synthesis
Title generation	Haiku	Simple task, fast

Cost Optimization

```
Before (Two Systems):
├── Chatbot always used Haiku
└── Agentic always used GPT-4/Sonnet

After (Unified Orchestrator):
├── 70% of requests → Haiku (simple + single_tool)
└── 30% of requests → Sonnet (complex only)

Result: ~40% cost reduction on AI API calls
```

MCP Tool Categories

Category	Tools	Examples
Invoices	9	search_invoices, get_overdue_trends
Customers	5	get_customer_profile, get_risk_analysis
Cases	6	search_cases, get_case_summary
Payment Arrangements	4	get_payment_plan, get_customer_plans
Remittances	4	get_remittance, get_customer_remittances
Workflows	8	get_case_workflow, get_company_workflows
Disputes	3	get_dispute, get_case_disputes
Followups	3	get_followup, get_source_followups

Total: 42 Tools

Database Models

ChatConversation

```
{
  userId: ObjectId,
  companyId: ObjectId,
  title: String,           // AI-generated
  messageCount: Number,
  isActive: Boolean,
  lastMessageAt: Date,
  expiresAt: Date,         // 30-day TTL
}
```

ChatMessage

```
{
  conversationId: ObjectId,
  role: "user" | "assistant",
  content: String,
  claudeMessage: Mixed,    // Full API response
  metadata: {
    toolCalls: Array,
    tokensUsed: Number,
    model: String,
  }
}
```

ChatUsage

```
{
  userId: ObjectId,
  companyId: ObjectId,
  date: String,            // YYYY-MM-DD
  successfulCount: Number, // Max 30/day
  failedCount: Number,
}
```

Security & Access Control

Authentication Flow

```
WebSocket Connection
    |
    v
JWT Token Validation
    |
    v
Company Subscription Check
├── Sandbox: ALLOW (bypass)
├── Free: DENY
└── Trial/Standard/Enterprise: ALLOW
    |
    v
Daily Rate Limit Check (30/day)
    |
    v
Process Request
```

Rate Limiting

- **Limit:** 30 successful chats per user per day
- **Reset:** Midnight UTC
- **Failed messages:** Tracked but don't count against limit

Environment Configuration

```
# AI Services
ANTHROPIC_API_KEY=sk-ant-...

# MCP Server
MCP_SERVER_URL=http://localhost:3002

# Database
MONGODB_URI=mongodb://...
REDIS_URL=redis://...

# Security
```

```
JWT_SECRET=...

# Feature Flags
ENABLE_AGENTIC=true
```

File Structure

```
src/services/
├── ai-chat-bot/
│   ├── orchestrator.js          # Unified orchestrator
│   ├── claude-service.js        # Legacy (still available)
│   ├── index.js                 # Exports
│   ├── websocket/
│   │   └── chat-handler.js      # WebSocket routing
│   ├── controllers/
│   │   └── chat-controller.js   # REST endpoints
│   ├── services/
│   │   ├── chat-history.service.js
│   │   └── chat-usage.service.js
│   └── models/
│       ├── ChatConversation.js
│       ├── ChatMessage.js
│       └── ChatUsage.js
│
├── mcp/
│   ├── unified-mcp-client.js    # Singleton MCP client
│   └── index.js                 # Exports
│
└── agentic/                     # Legacy (can be removed)
    ├── agentic-engine.js
    ├── llm-provider.js
    └── skills-manager.js
```

Migration Notes

What Changed

1. **New orchestrator.js** - Single entry point for all AI
2. **chat-handler.js** - Now uses orchestrator instead of claudeService
3. **index.js** - Exports orchestrator

What Stayed the Same

- claude-service.js - Still available for direct use
- unified-mcp-client.js - No changes
- All database models - No changes
- All MCP tools - No changes

Rollback

To rollback, change chat-handler.js:

```
// From:
const { getOrchestrator } = require("../orchestrator");
// To:
const claudeService = require("../claude-service");
```

Benefits of Unified Architecture

Benefit	Description
Single Entry Point	All AI requests go through one orchestrator
Smart Routing	Automatically picks the right model for the task
Cost Optimization	Uses cheap model for simple tasks
Better UX	Faster responses for simple queries
Maintainability	One codebase instead of two
Scalability	Easy to add new complexity levels

References

- Anthropic - Building Effective Agents
- Anthropic - Multi-Agent Research System
- Model Context Protocol Specification
- Agentic AI Foundation

Document generated: February 2026 Architecture Version: 2.0 (Unified Orchestrator)